

학사관리시스템 — 설계·구축·조회 통합 리포트

종합실습 1 — 학사관리시스템 DB 설계 → 구축 → 조회 대상 DBMS: PostgreSQL
17 / 설계도구: ERDCloud·dbdiagram.io

이 문서는 제1부 요구사항 분석 및 설계 방향과 제2부 **SQL** 구현(구축·조회) 리포트를 하나로 합친 통합본이다. 제1부에서 도출한 엔티티·관계·제약·조회 시나리오가 제2부의 DDL·DML·조회 쿼리로 어떻게 구현되는지 end-to-end로 대응된다.

목차

제1부 — 요구사항 분석 (설계 방향)

1. 시스템 개요 및 설계 목적
2. 설계 범위 (Scope)
3. 기능 요구사항 (Functional Requirements)
4. 비기능 요구사항
5. 핵심 엔티티 도출 (ERD 연결)
6. 주요 관계 및 카디널리티
7. 주요 데이터 무결성 제약
8. 주요 조회 시나리오 (실습 과제 매핑)
9. 설계 산출물 체크리스트

제2부 — SQL 구현 (구축·조회)

- 문항 1. CREATE DATABASE / CREATE SCHEMA
 - 문항 2. ERD 설계
 - 문항 3. CREATE TABLE — 제약조건 포함 DDL
 - 문항 4. INSERT — 테이블별 10건 이상
 - 문항 5. SELECT + WHERE + ORDER BY 기초 조회
 - 문항 6. COALESCE / CASE WHEN / 날짜 함수 활용
 - 문항 7. 수강신청 교차 테이블 JOIN 조회
-
-

제1부 — 요구사항 분석 (설계 방향)

1. 시스템 개요 및 설계 목적

대학의 학사 업무 전 주기(입학 이후 학적·수강·성적·등록)를 지원하는 학사관리시스템의 데이터베이스를 설계·구축·조회한다.

분석 대상이 된 실제 대학 포털 메뉴는 자기정보관리부터 국외교류까지 **100개** 이상의 기능으로 구성되어 있으나, 이를 그대로 구현하는 것은 실습 범위를 초과한다. 따라서 학사관리의 핵심 도메인 **5개**를 선정하여 정규화된 관계형 스키마로 설계하고, **ERD → DDL → DML → 조회 쿼리**로 이어지는 **end-to-end** 구현을 목표로 한다.

2. 설계 범위 (Scope)

메뉴 전수 분석 결과를 학사관리 핵심 / 경계·확장 두 축으로 분류하여 범위를 확정한다.

2.1 In-Scope — 핵심 5개 도메인 (구현 대상)

No	도메인	대응 포털 메뉴 (요약)	포함 이유
①	학적관리	자기정보관리, 휴/복학신청, 학적변동(조기졸업· 재입학·전과·자퇴)	학적의 기초 신상·신분변동
②	전공·이수과정	전공/교직관리(전공· 이중전공·부전공·마 이크로디그리·교직이 수)	교육과정 편성·이수의 기준
③	수업·수강관리	수강신청, 개인/강의시간표, 강의계획서, 재수강	학기별 강좌 운영과 수강
④	성적·졸업관리	성적취득현황, 성적추이, 교양영역별 취득, 졸업요건	성적 산출·졸업사정

No	도메인	대응 포털 메뉴 (요약)	포함 이유
⑤	등록·장학관리	등록내역, 교육비납입, 장학내역, 분할납부, 계좌등록	등록금 정산과 장학 수혜

2.2 Out-of-Scope — 경계·확장 영역 (제외 및 근거)

항목	성격	제외 근거
국외교류(교류선발·자비유학·7+1파견)	프로그램 운영	선발·파견은 승인·상태전이 중심 워크플로우로 별도 프로세스 모델링 필요. 단, (교류)이수과목 학점인정 → 졸업요건 연계 부분만 개념적으로 반영
계절학기 환불	등록의 파생	정규 등록과 별개의 환불 정산 로직이 필요해 등록 도메인의 확장으로 분리
공로·봉사장학생	근로·봉사 운영	명목상 장학이나 실질은 교내 근로 배정·근태 관리로 인사/근로 성격에 가까움

범위 확정 원칙: 상태 전이·승인 흐름이 강한 "프로세스 운영형" 기능은 제외하고, 조회·집계 중심으로 데이터를 축적하는 "마스터·이력형" 기능을 우선 구현한다.

3. 기능 요구사항 (Functional Requirements)

FR-1. 학적관리

- **FR-1.1** 학생의 기본 신상정보(학번, 성명, 생년월일, 성별, 연락처, 이메일, 주소)를 등록·조회·수정한다.
- **FR-1.2** 학생의 소속 학과, 학년, 입학일, 학적상태(재학·휴학·졸업·자퇴)를 관리한다.
- **FR-1.3** 개인정보 활용동의 여부를 관리한다.
- **FR-1.4** 학적변동(휴학·복학·재입학·전과·자퇴·조기졸업) 이력을 변동유형·변동일자·사유와 함께 기록·조회한다.

FR-2. 전공·이수과정 관리

- **FR-2.1** 학과/전공 마스터(학과코드, 학과명, 계열)를 관리한다.
- **FR-2.2** 학생의 주전공 배정 및 다중전공(이중전공·부전공·전공심화·마이크로디그리) 이수 관계를 관리한다.
- **FR-2.3** 교직·지역학·응급처치·성인지 등 부가 이수과정의 이수여부를 관리한다. (선택 구현)

FR-3. 수업·수강관리

- **FR-3.1** 교과목 마스터(과목코드, 과목명, 학점, 이수구분: 전공/교양/일반)를 관리한다.
- **FR-3.2** 학기별 개설강좌(분반)에 담당교수·강의시간·강의실을 편성한다.
- **FR-3.3** 학생이 개설강좌를 수강신청한다(학생 N : M 개설강좌, 신청일자·신청상태 포함).
- **FR-3.4** 개인 시간표 및 강의별 수강 명단을 조회한다.

FR-4. 성적·졸업관리

- **FR-4.1** 수강 결과 성적(점수·등급)과 재수강 여부를 기록한다.
- **FR-4.2** 학기별·누적 GPA와 성적추이를 조회한다.
- **FR-4.3** 이수구분별·교양영역별 취득학점을 집계한다.
- **FR-4.4** 졸업요건(총 이수학점·영역별 최소학점) 대비 이수현황을 조회한다.

FR-5. 등록·장학관리

- **FR-5.1** 학기별 등록 내역(고지금액·납부금액·납부일·분할납부 여부)을 관리한다.
- **FR-5.2** 장학 유형과 학생별 수혜내역(장학금액·수혜학기)을 관리한다.
- **FR-5.3** 환불·이체용 계좌번호를 등록·조회한다.

4. 비기능 요구사항 (요약)

- 무결성: PK/FK·UNIQUE·CHECK 제약으로 중복·이상 데이터 방지 (정규화 3NF 기준).
 - 일관성: 코드성 값(학적상태·이수구분·성적등급)은 CHECK 도메인 또는 코드 테이블로 통제.
 - 확장성: 학기·연도 단위로 수강·등록 데이터가 누적되므로 (학생, 학기) 기준 조회에 대비한 인덱스 설계.
 - 표준성: 금액은 NUMERIC, 일시는 DATE/TIMESTAMP로 타입 통일.
-

5. 핵심 엔티티 도출 (ERD 연결)

기능 요구사항으로부터 다음 엔티티를 도출한다.

엔티티	유형	주요 속성	근거 FR
학과 department	강한	학과코드(PK), 학과명, 계열	FR-2.1
학생 student	강한	학번(PK), 성명, 생년월일, 학년, 학적상태, 입학일, 학과코드(FK)	FR-1.1~1.3
학적변동 status_change	약한	(학번+변동일자) PK, 변동유형, 사유	FR-1.4
전공이수 student_major	교차	(학번+학과코드) PK, 전공구분(주/복수/부)	FR-2.2
교수 professor	강한	교수번호(PK), 성명, 소속학과(FK)	FR-3.2
교과목 course	강한	과목코드(PK), 과목명, 학점, 이수구분	FR-3.1
개설강좌 offering	강한	개설번호(PK), 과목코드(FK), 교수번호(FK), 학기, 분반, 강의시간	FR-3.2
수강신청 enrollment	교차	(학번+개설번호) PK, 신청일자, 성적점수, 재수강여부	FR-3.3, FR-4.1
등록 registration	강한	(학번+학기) PK, 고지금액, 납부금액, 납부일, 분할여부	FR-5.1
장학내역 scholarship	강한	장학번호(PK), 학번(FK), 장학유형, 금액, 수혜학기	FR-5.2

최소 구축 세트(권장): 학과·학생·교과목·개설강좌·수강신청·등록·장학 (7개) —
실습의 JOIN/집계 요구를 모두 충족. 확장 세트: 학적변동·전공이수·교수 추가 시
학적·전공 도메인까지 완결.

6. 주요 관계 및 카디널리티

관계	카디널리티	설명
학과 — 학생	1 : N	한 학과에 여러 학생 소속
학과 — 교과목	1 : N	학과가 교과목 개설 주체
학생 — 학적변동	1 : N	학생의 휴/복학 등 이력 누적
학생 — 전공	N : M	다중전공 → student_major 교차
교수 — 개설강좌	1 : N	한 교수가 여러 강좌 담당
교과목 — 개설강좌	1 : N	한 과목이 학기별 여러 분반으로 개설
학생 — 개설강좌	N : M	수강신청 enrollment 교차 테이블 (성적 속성 보유)
학생 — 등록	1 : N	학기마다 등록 1건
학생 — 장학내역	1 : N	학기별 장학 수혜

7. 주요 데이터 무결성 제약

- 학번·과목코드·개설번호: **UNIQUE NOT NULL**
 - 수강신청: **PRIMARY KEY**(학번, 개설번호)로 동일 강좌 중복신청 방지
 - 성적점수: **CHECK (score BETWEEN 0 AND 100)**
 - 학적상태: **CHECK (status IN ('재학', '휴학', '졸업', '자퇴'))**
 - 학점: **CHECK (credit > 0)**
 - **FK 삭제정책**: 학생 삭제 시 수강·등록·장학은 **ON DELETE CASCADE**
-

8. 주요 조회 시나리오 (실습 과제 매핑)

제출 요건(WHERE·ORDER BY·CASE WHEN·COALESCE·날짜함수·교차테이블 JOIN)을 충족하는 대표 쿼리 시나리오. 각 시나리오는 제2부의 문항 5~7에서 실제 쿼리로 구현된다.

#	조회 시나리오	핵심 SQL 요소	구현 위치
Q1	재학생 명단을 학번순으로 조회	WHERE status='재학' + ORDER BY	문항 5 (Q5-1)
Q2	학과별 재학생 수 집계	GROUP BY + COUNT	문항 4 검증 등
Q3	특정 학기 수강신청 내역(학생·개설강좌· 교과목)	3-테이블 JOIN (교차테이블)	문항 7
Q4	성적점수를 A/B/C/D 등급으로 변환 표시	CASE WHEN	문항 7
Q5	장학 미수혜자를 '해당없음'으로 표기	COALESCE + LEFT JOIN	문항 6·7
Q6	입학 후 경과연수·휴학기간 산출	날짜 함수(AGE, CURRENT_DATE)	문항 6
Q7	학과별 GPA 상위 학생 추출	집계 + 정렬(심화 시 Window Function)	확장

9. 설계 산출물 체크리스트 (제출물 대응)

- ☒ 학사관리시스템 요구사항(본 문서 제1부 = 설계 방향)
- ☒ ERD — 범례 포함, 관계선 겹침 없이 작성 → 제2부 문항 2
- ☒ DDL — 제약조건(PK/FK/UNIQUE/CHECK) 포함 CREATE TABLE → 제2부 문항 3
- ☒ DML — 테이블별 샘플 데이터 10건 이상 INSERT → 제2부 문항 4
- ☒ 조회 — Q1Q7 쿼리와 실행결과 화면 → 제2부 문항 5·7
- ☐ PostgreSQL 접속 결과 화면 (psql 또는 DBeaver)

제2부 — SQL 구현 (구축·조회)

문항 1. CREATE DATABASE / CREATE SCHEMA

1-1. 데이터베이스 생성

-- =====

-- 목적: 학사관리시스템 데이터베이스 생성

-- · ENCODING / LOCALE 은 생성 후 변경 불가 → 최초에 확정

-- · ICU 제공자: OS 업그레이드 시 정렬 규칙 변경(인덱스 손상) 방지

-- · template0: 로케일을 바꾸려면 깨끗한 템플릿이 필요

-- =====

CREATE DATABASE skala_db

WITH OWNER = muna

TEMPLATE = template0

ENCODING = 'UTF8'

LOCALE_PROVIDER = 'icu'

ICU_LOCALE = 'ko-KR'

LC_COLLATE = 'en_US.UTF-8'

LC_CTYPE = 'en_US.UTF-8'

CONNECTION LIMIT = -1;

COMMENT ON DATABASE skala_db IS 'SKALA SQL 종합실습 1 — 학사관리시스템';

설계 근거 (리포트에 서술)

옵션	선택 이유
TEMPLATE = template0	로케일 변경 시 필수. template1 은 기존 객체까지 복사된다
LOCALE_PROVIDER = 'icu'	libc는 OS 버전업 시 정렬 순서가 바뀌어 기존 인덱스가 조용히 깨진다. ICU는 PostgreSQL이 버전을 추적한다
ICU_LOCALE = 'ko-KR'	한글 가나다순 정렬
ENCODING = 'UTF8'	생성 후 변경 불가 — 변경하려면 dump → drop → create → restore

1-2. 생성 검증

-- 목적: 인코딩·콜레이션·제공자가 의도대로 지정되었는지 검증

SELECT datname,

pg_encoding_to_char(encoding) AS encoding,

datlocprovider AS provider,

datcollate,

datlocale AS icu_locale

FROM pg_database

WHERE datname = 'skala_db';

PostgreSQL 17에서 **pg_database.daticulocale** 이 **datlocale** 로 이름이 바뀌었다 (builtin 로케일 제공자 추가로 "ICU 전용" 이름이 부적합해짐). 구버전 예제를 그대로 쓰면 **column "daticulocale" does not exist** 오류가 난다.

1-3. 스키마 생성

```
\c skala_db
```

```
-- =====
```

```
-- 목적: 애플리케이션 전용 스키마 생성
```

```
-- · public 을 쓰지 않는 이유
```

```
-- ① 확장(extension)이 기본 설치되는 공간이라 업무 테이블과 섞임
```

```
-- ② 이름이 의미를 전달하지 못함 (app.students = "앱 업무 테이블"이 드러남)
```

```
-- =====
```

```
CREATE SCHEMA IF NOT EXISTS app AUTHORIZATION muna;
```

```
COMMENT ON SCHEMA app IS '학사관리 업무 테이블';
```

```
-- 목적: 이 DB 접속 시 app 스키마를 기본 탐색 경로로 사용
```

```
ALTER DATABASE skala_db SET search_path TO app, public;
```

```
SET search_path TO app, public;
```

설계 근거 — **audit** 스키마는 만들지 않았다. 이번 설계의 12개 테이블은 전부 업무 테이블이고, 감사 로그 요구사항이 없다. **status_changes**는 이름이 "이력"이지만 학생이 포털에서 직접 조회하는 업무 데이터이므로 **app**에 속한다. 쓰지 않을 빈 스키마를 미리 만들면 설계 근거를 설명할 수 없다.

1-4. 검증

```
-- 목적: 스키마 생성 및 현재 접속 컨텍스트 확인
```

```
SELECT current_database() AS db,
```

```
       current_schema() AS 현재스키마,
```

```
       current_user     AS 접속계정;
```

```
\dn
```

문항 2. ERD 설계

SQL 없음. **ERD** 이미지 + 범례 첨부.

- 도구: dbdiagram.io (DBML 소스: `skala_academic.dbml`)
- 테이블 12 · 관계 15 · 복합 PK 3
- 범례는 별도 Table 객체로 작성하여 export 에 포함되도록 처리
- 관계선 겹침 없이 수동 배치

문항 3. CREATE TABLE — 제약조건 포함 DDL

전문은 첨부 파일 `01_ddl.sql` 참조. 리포트 본문에는 아래 요약표와 대표 테이블 2개를 싣고, 전문은 부록으로 첨부하는 구성을 권장한다.

3-1. 생성 순서 (FK 의존성 위상 정렬)

1단계 (독립) `semesters`, `departments`, `students`

2단계 (1단계 참조) `professors`, `courses`, `student_majors`, `status_changes`,

`registrations`, `scholarships`

3단계 (2단계 참조) `offerings`

4단계 (3단계 참조) `offering_schedules`, `enrollments`

3-2. 대표 테이블 ① `students` — 마스터

-- 목적: 학생 마스터.

-- 학과 FK 를 두지 않는다 — 전공 정보는 `student_majors` 가 단일 진실 원천.

-- 양쪽에 저장하면 전과 시 한쪽만 갱신되는 갱신 이상(3NF 위반)이 발생한다.

CREATE TABLE app.students (

```
id          BIGINT    GENERATED ALWAYS AS IDENTITY,

student_no  VARCHAR(10) NOT NULL,

name        VARCHAR(50) NOT NULL,

birth_date  DATE      NOT NULL,

gender      VARCHAR(1),

phone       VARCHAR(20),

email       VARCHAR(255) NOT NULL,

address     VARCHAR(200),

admitted_on DATE      NOT NULL,

grade       SMALLINT  NOT NULL,

status      VARCHAR(10) NOT NULL DEFAULT '재 학',

privacy_agreed BOOLEAN NOT NULL DEFAULT false,

refund_account VARCHAR(50),

created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

CONSTRAINT pk_students      PRIMARY KEY (id),

CONSTRAINT uq_students_student_no UNIQUE (student_no),

CONSTRAINT uq_students_email  UNIQUE (email),

CONSTRAINT ck_students_gender  CHECK (gender IN ('M','F')),

CONSTRAINT ck_students_grade   CHECK (grade BETWEEN 1 AND 4),

CONSTRAINT ck_students_status  CHECK (status IN ('재 학','휴 학','졸업','자퇴')),

CONSTRAINT ck_students_birth   CHECK (birth_date < admitted_on)
```

);

3-3. 대표 테이블 ② **enrollments** — 교차 테이블

-- 목적: 수강신청 교차 테이블. 학생 N:M 개설강좌 + 성적 속성.

-- PK(student_id, offering_id) 가 곧 "동일 강좌 중복신청 불가" 제약이다.

-- score NULL = 미채점. 성적등급은 저장하지 않고 조회 시 CASE WHEN 으로 파생.

```
CREATE TABLE app.enrollments (  
  
    student_id BIGINT    NOT NULL,  
  
    offering_id BIGINT    NOT NULL,  
  
    enrolled_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  
    status    VARCHAR(10) NOT NULL DEFAULT '신청',  
  
    score     NUMERIC(5,2),  
  
    is_retake BOOLEAN    NOT NULL DEFAULT false,  
  
    CONSTRAINT pk_enrollments PRIMARY KEY (student_id, offering_id),  
  
    CONSTRAINT fk_enrollments_student_id FOREIGN KEY (student_id)  
  
        REFERENCES app.students (id) ON DELETE CASCADE ON UPDATE CASCADE,  
  
    CONSTRAINT fk_enrollments_offering_id FOREIGN KEY (offering_id)  
  
        REFERENCES app.offerings (id) ON DELETE CASCADE ON UPDATE CASCADE,  
  
    CONSTRAINT ck_enrollments_status CHECK (status IN ('신청','확정','취소')),  
  
    CONSTRAINT ck_enrollments_score CHECK (score BETWEEN 0 AND 100)  
  
);
```

3-4. 업무규칙 → 제약조건 매핑 (리포트 핵심 표)

#	업무 규칙	구현 방식
BR-1	학생은 주전공을 정확히 1개 가진다	부분 유니크 인덱스
BR-2	같은 학생이 같은 강좌를 중복 신청할 수 없다	복합 PK (<i>student_id</i> , <i>offering_id</i>)
BR-3	성적은 0~100	CHECK
BR-4	학적상태 ∈ {재학, 휴학, 졸업, 자퇴}	CHECK
BR-5	학점 > 0	CHECK
BR-6	납부금액 ≤ 고지금액	테이블 레벨 CHECK
BR-7	종강일 > 개강일	테이블 레벨 CHECK
BR-8	교양과목만 교양영역을 가진다	테이블 레벨 조건부 CHECK
BR-9	학생 삭제 시 부속 데이터 동반 삭제	ON DELETE CASCADE
BR-10	학과에 소속이 있으면 삭제 차단	ON DELETE RESTRICT

BR-1 — 일반 **UNIQUE**로 표현할 수 없는 제약

-- 일반 **UNIQUE(student_id)** 는 부전공까지 막으므로 부분 인덱스가 필요하다

```
CREATE UNIQUE INDEX uq_student_majors_primary
```

```
ON app.student_majors (student_id)
```

```
WHERE major_type = '주전공';
```

BR-8 — 두 컬럼을 동시에 보는 조건부 **CHECK**

```
CONSTRAINT ck_courses_liberal_area CHECK (
```

```
(course_type = '교양' AND liberal_area IS NOT NULL)

OR (course_type <> '교양' AND liberal_area IS NULL)

)
```

3-5. 인덱스 설계

-- 목적: FK 인덱스 + 조회 패턴 인덱스

-- · PostgreSQL 은 FK 에 인덱스를 자동 생성하지 않는다 (PK/UNIQUE 만 자동)

-- · PK/UNIQUE 인덱스의 선두 컬럼으로 이미 커버되는 FK 는 만들지 않는다

```
CREATE INDEX idx_enrollments_offering_id ON app.enrollments (offering_id);
```

```
CREATE INDEX idx_students_status ON app.students (status);
```

-- (이하 01_ddl.sql 참조)

설계 근거 **2**가지 (리포트 서술용)

1. **PostgreSQL**은 **FK**에 인덱스를 자동 생성하지 않는다. 자동인 것은 **PK**와 **UNIQUE**뿐이다. 인덱스 없는 **FK**는 **JOIN**이 느릴 뿐 아니라 부모 삭제 시 **CASCADE/RESTRICT** 검사가 자식 테이블을 전체 스캔한다.
2. 복합 **PK**는 왼쪽 접두사로만 쓰인다. **PK(student_id, offering_id)**는 **WHERE offering_id = ?**("이 강좌의 수강생 명단")에 사용되지 않으므로 별도 인덱스가 필요하다. 반대로 **UNIQUE** 제약의 선두 컬럼과 겹치는 **FK** 인덱스 3개는 중복이라 생략했다.

3-6. 검증

-- 목적: 테이블 12개 생성 확인

```
SELECT table_name AS 테이블
```

```
FROM information_schema.tables
```

```
WHERE table_schema = 'app' AND table_type = 'BASE TABLE'
```

```
ORDER BY table_name;
```

-- 목적: 제약조건 종류별 집계

```
SELECT CASE c.contype WHEN 'p' THEN 'PRIMARY KEY'
        WHEN 'f' THEN 'FOREIGN KEY'
        WHEN 'u' THEN 'UNIQUE'
        WHEN 'c' THEN 'CHECK' END AS 제약종류,
        COUNT(*) AS 개수
FROM pg_constraint c
JOIN pg_namespace n ON n.oid = c.connamespace
WHERE n.nspname = 'app'
GROUP BY c.contype
ORDER BY c.contype;
```

기대 결과

항목	개수
테이블	12
PRIMARY KEY	12
FOREIGN KEY	15 (CASCADE 7 · RESTRICT 7 · SET NULL 1)
UNIQUE	11
CHECK	25
인덱스(명시)	11

문항 4. INSERT — 테이블별 10건 이상

전문은 첨부 파일 **02_dml_insert.sql** 참조.

4-1. FK 연결 방식 (리포트 핵심 설명)

PK가 **GENERATED ALWAYS AS IDENTITY**이므로 **id**를 직접 지정할 수 없다. 따라서 **FK**는 자연키로 조회해서 대리키를 얻는 방식으로 연결한다.

-- 목적: 교수 마스터 적재. **department_id** 는 학과코드로 조회해서 연결한다.

```
INSERT INTO app.professors (professor_no, name, department_id, hired_on, email)
```

```
SELECT v.professor_no, v.name, d.id, v.hired_on, v.email
```

```
FROM (VALUES
```

```
  ('P001', '김정보', 'CSE', DATE '2010-03-01', 'kim.jb@skala.ac.kr'),
```

```
  ('P005', '정화학', 'CHE',   '2018-03-01', NULL) -- email 선택 입력
```

```
  -- ... 12건
```

```
  ) AS v(professor_no, name, dept_code, hired_on, email)
```

```
JOIN app.departments d ON d.code = v.dept_code;
```

id를 1, 2, 3으로 가정해 하드코딩하면 재실행 시 시퀀스가 밀려 전부 깨진다.

주의점 — **VALUES** 안의 따옴표 문자열은 타입이 **unknown**이고 서브쿼리 경계를 넘으면 **text**로 확정된다. **text** → **date**는 대입 캐스트가 없어 **INSERT**가 실패하므로, 날짜·시각 컬럼은 첫 행에 **DATE '...' / TIMESTAMPTZ '...'**로 타입을 앵커한다.

4-2. 적재 건수

테이블	건수	테이블	건수
semesters	12	offerings	20
departments	10	offering_schedules	30

테이블	건수	테이블	건수
students	20	enrollments	61
professors	12	registrations	30
courses	16	scholarships	16
student_majors	28	status_changes	12

4-3. 의도적으로 심은 NULL (조회 실습의 재료)

케이스	건수	쓰이는 곳
<code>enrollments.score</code> NULL — 미채점	12	문항 7 CASE WHEN
<code>offerings.professor_id</code> NULL — 담당교수 미배정	2	문항 7 LEFT JOIN + COALESCE
<code>registrations.paid_on</code> NULL — 미납	3	문항 5 IS NULL
<code>status_changes.reason</code> NULL — 사유 미기재	3	COALESCE
<code>students.phone / address</code> NULL — 선택 입력	5	문항 5·6
<code>courses.department_id</code> NULL — 교양과목	6	문항 5
장학 미수혜 — 행 자체가 없음	11명	LEFT JOIN + COALESCE(SUM, 0)

NULL은 값 누락이 아니라 업무적으로 의도된 미확정 상태다. 마지막 항목은 특히 중요한데, "장학 미수혜"는 컬럼 NULL이 아니라 `scholarships`에 행 자체가 없는 것이다. 컬럼 NULL과 행 부재는 다른 개념이며 처리 방법도 다르다.

4-4. 검증

-- 목적: 테이블별 적재 건수 확인 (전 테이블 10건 이상이어야 함)

```
SELECT '01 semesters' AS 테이블, COUNT(*) AS 건수 FROM app.semesters
```

```

UNION ALL SELECT '02 departments',    COUNT(*) FROM app.departments
UNION ALL SELECT '03 students',       COUNT(*) FROM app.students
UNION ALL SELECT '04 professors',     COUNT(*) FROM app.professors
UNION ALL SELECT '05 courses',        COUNT(*) FROM app.courses
UNION ALL SELECT '06 student_majors',  COUNT(*) FROM app.student_majors
UNION ALL SELECT '07 status_changes',  COUNT(*) FROM app.status_changes
UNION ALL SELECT '08 offerings',       COUNT(*) FROM app.offerings
UNION ALL SELECT '09 offering_schedules', COUNT(*) FROM app.offering_schedules
UNION ALL SELECT '10 enrollments',     COUNT(*) FROM app.enrollments
UNION ALL SELECT '11 registrations',   COUNT(*) FROM app.registrations
UNION ALL SELECT '12 scholarships',    COUNT(*) FROM app.scholarships
ORDER BY 1;

```

-- 목적: BR-1 검증 — 모든 학생이 주전공을 정확히 1개씩 보유하는가

```

SELECT COUNT(*) AS 학생수,
        COUNT(*) FILTER (WHERE 주전공수 = 1) AS 주전공_정상,
        COUNT(*) FILTER (WHERE 주전공수 <> 1) AS 주전공_이상
FROM (SELECT s.id,
        COUNT(*) FILTER (WHERE sm.major_type = '주전공') AS 주전공수
FROM app.students s
LEFT JOIN app.student_majors sm ON sm.student_id = s.id
GROUP BY s.id) t;

```

문항 5. SELECT + WHERE + ORDER BY 기초 조회

Q5-1. 재학생 명단을 학번순으로 조회

-- 목적: 가장 기본적인 등가 조건(=) 필터와 오름차순 정렬

```
SELECT student_no AS 학번,
```

```
       name      AS 성명,
```

```
       grade     AS 학년,
```

```
       email     AS 이메일
```

```
FROM app.students
```

```
WHERE status = '재학'
```

```
ORDER BY student_no;
```

기대: **15건** (전체 20명 - 휴학 2 - 졸업 2 - 자퇴 1)

Q5-2. 3·4학년 재학생을 학년 내림차순, 성명 가나다순으로 조회

-- 목적: IN 목록 조건 + AND 결합 + 다중 컬럼 정렬(방향 혼합)

```
SELECT student_no AS 학번,
```

```
       name      AS 성명,
```

```
       grade     AS 학년,
```

```
       status    AS 학적상태
```

```
FROM app.students
```

```
WHERE grade IN (3, 4)
```

AND status = '재학'

ORDER BY grade DESC, name;

Q5-3. 연락처 또는 주소가 미기재된 학생 조회

-- 목적: NULL 판별. IS NULL 을 써야 하는 이유를 보여준다.

SELECT student_no AS 학번,

name AS 성명,

phone AS 연락처,

address AS 주소

FROM app.students

WHERE phone IS NULL

OR address IS NULL

ORDER BY student_no;

기대: 5건 설명 포인트 — WHERE phone = NULL 은 오류 없이 0건을 반환한다. NULL은 "알 수 없는 값"이라 = 비교 결과가 참도 거짓도 아닌 unknown이 되고, WHERE는 참인 행만 통과시키기 때문이다. 오류가 나지 않아 조용히 틀린 결과를 주는 쪽이 더 위험하다. 📸 캡처 9

문법 요소 커버 표 (리포트에 함께 게재)

요소	Q5-1	Q5-2	Q5-3
SELECT 컬럼 명시 (*미사용)	•	•	•
목적 주석	•	•	•
WHERE 등가 조건 =	•	•	
WHERE 목록 조건 IN + AND		•	

요소	Q5-1	Q5-2	Q5-3
WHERE NULL 판별 IS NULL + OR			•
ORDER BY 단일 정렬	•		•
ORDER BY 다중 컬럼 + 방향 혼합		•	

문항 6. COALESCE / CASE WHEN / 날짜 함수 활용

-- 목적: 재적 학생의 연락처 보완 표시 · 학적 파생구분 · 입학 후 경과기간 조회

-- 함수: COALESCE / CASE WHEN / TO_CHAR / DATE_TRUNC / EXTRACT / AGE / INTERVAL

-- 주의: WHERE 에는 SELECT 별칭(학적구분)을 쓸 수 없어 원본 컬럼 **status** 를 쓴다

```

SELECT student_no                AS 학번,

       name                      AS 성명,

       COALESCE(phone, '미기재') AS 연락처,

       TO_CHAR(admitted_on, 'YYYY"년" MM"월"') AS 입학시기,

       DATE_TRUNC('month', admitted_on)::date AS 입학월초,

       EXTRACT(YEAR FROM AGE(CURRENT_DATE, admitted_on)) AS 재학연차,

       CASE WHEN status = '재 학' AND grade = 4 THEN '졸업예정'

            WHEN status = '재 학'          THEN '재 학중'

            WHEN status = '휴 학'          THEN '휴 학중'

            ELSE                            '졸업'

       END                        AS 학적구분,

```

```

CASE WHEN admitted_on <= CURRENT_DATE - INTERVAL '4 years'

      THEN '수료연한 도래'

      ELSE '재학기간 내'

END                                     AS 수료연한

FROM app.students

WHERE status <> '자퇴'

ORDER BY admitted_on, student_no;

```

기대: **19**건 (전체 20명 - 자퇴 1)

요구 요소 커버

요소	사용처
COALESCE	phone NULL → '미기재' (5건 해당)
CASE WHEN	학적구분 4분기 / 수료연한 2분기
TO_CHAR	날짜 포매팅 (2022년 03월)
DATE_TRUNC	월 단위 절삭
EXTRACT + AGE	입학 후 경과 연수
INTERVAL	CURRENT_DATE - INTERVAL '4 years'

설명 포인트 — WHERE에서 SELECT 별칭을 쓸 수 없는 이유는 실행 순서 때문이다.

FROM → WHERE → SELECT → ORDER BY

WHERE가 실행되는 시점에는 SELECT의 별칭이 아직 만들어지지 않았다. 그래서 WHERE 학적구분 = '재학중'은 column "학적구분" does not exist 오류가 나고, ORDER BY 학적구분은 정상 동작한다.

문항 7. 수강신청 교차 테이블 JOIN 조회

-- 목적: 학생 N:M 개설강좌 관계를 교차 테이블 **enrollments** 로 연결하여

-- "누가 · 어느 학기에 · 무슨 과목을 · 누구에게 · 몇 점으로" 수강했는지 조회

```
SELECT sm.year::text || '-' || sm.term    AS 학기,

      s.student_no                        AS 학번,

      s.name                             AS 성명,

      c.code                             AS 과목코드,

      c.title                            AS 과목명,

      c.credits                          AS 학점,

      o.section                          AS 분반,

      COALESCE(p.name, '미배정')         AS 담당교수,

      e.score                            AS 성적,

      CASE WHEN e.score IS NULL THEN '미채점'

      WHEN e.score >= 90 THEN 'A'

      WHEN e.score >= 80 THEN 'B'

      WHEN e.score >= 70 THEN 'C'

      WHEN e.score >= 60 THEN 'D'

      ELSE                               'F'

      END                                AS 등급,

      CASE WHEN e.is_retake THEN '재수강' ELSE '-' END AS 재수강

FROM app.enrollments e
```



```

JOIN app.students s ON s.id = e.student_id -- 교차 → 학생

JOIN app.offerings o ON o.id = e.offering_id -- 교차 → 개설강좌

JOIN app.courses c ON c.id = o.course_id -- 개설강좌 → 교과목

JOIN app.semesters sm ON sm.id = o.semester_id -- 개설강좌 → 학기

LEFT JOIN app.professors p ON p.id = o.professor_id -- 선택 관계이므로 LEFT

WHERE sm.year >= 2025

AND sm.term = '1'

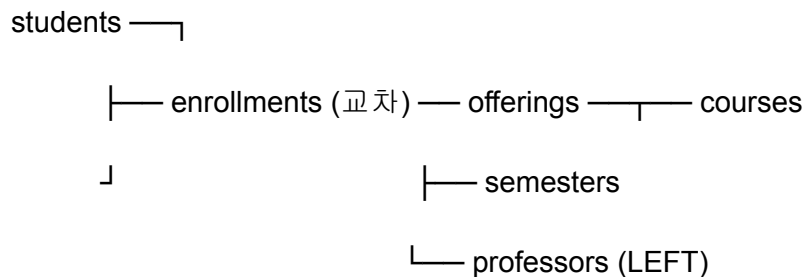
AND e.status = '확정'

ORDER BY sm.year, c.code, s.student_no;

기대: 21건 (2025-1 확정 11 + 2026-1 확정 10)

```

JOIN 구조



설명 포인트 2가지 (리포트 핵심)

① 교수만 LEFT JOIN인 이유

`offerings.professor_id`는 NULL 허용이다. INNER JOIN으로 붙이면 담당교수 미배정 강좌의 수강생 4명이 결과에서 통째로 사라진다. 개념설계 단계에서 "교수는 강좌를 0개 이상 담당한다"고 참여도를 선택으로 정한 결정이, 논리설계에서 ON DELETE SET NULL을 가능하게 했고, 조회 단계에서 JOIN 종류까지 규정한 것이다. 나머지 4개 FK는 전부 NOT NULL이므로 INNER JOIN이 맞다.

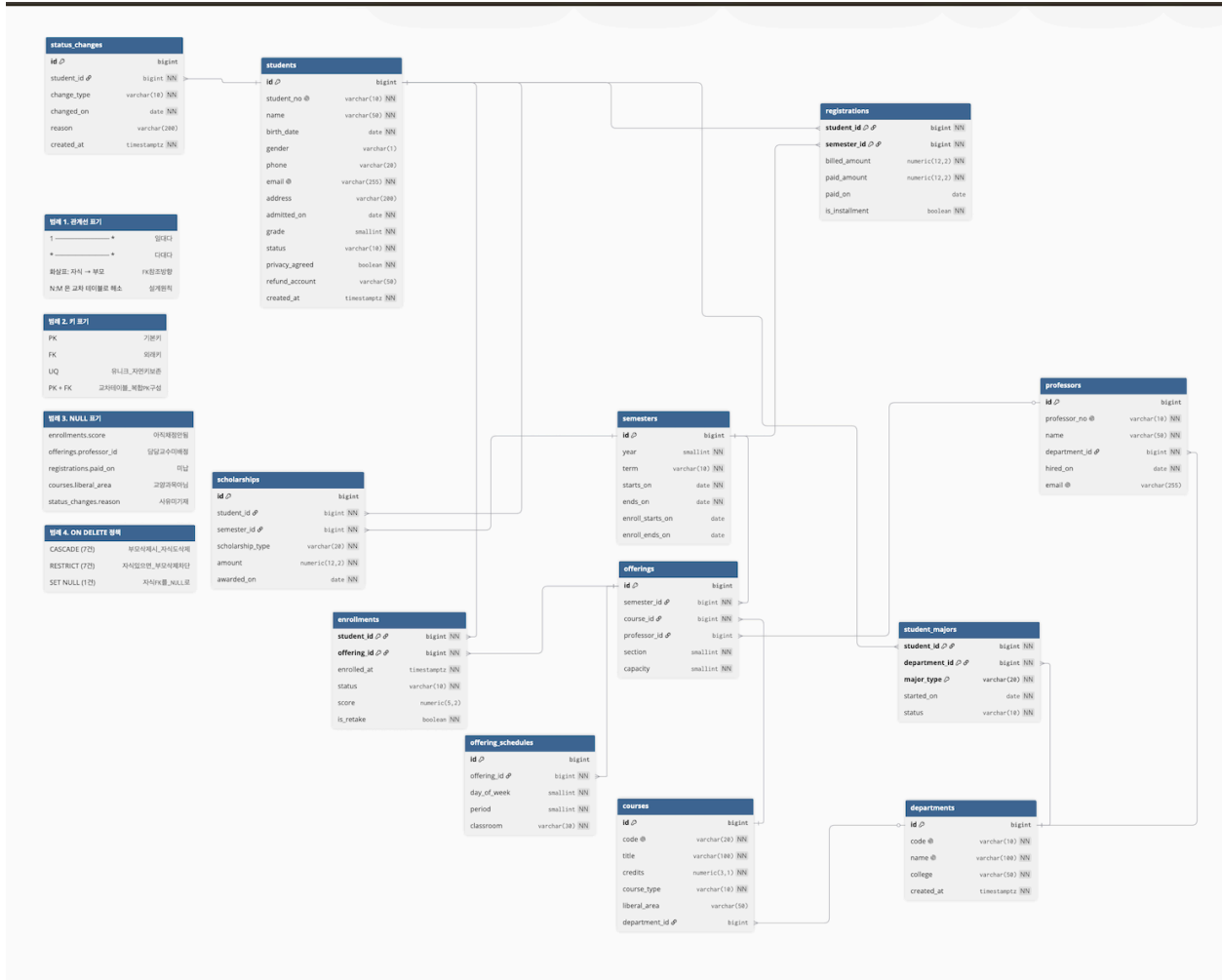
② 수강신청은 **courses**가 아니라 **offerings**에 걸린다

enrollments.offering_id → offerings.course_id → courses

한 단계를 더 거친다. **courses**는 카탈로그(불변), **offerings**는 학기마다 생성되는 실체다. 이 분리 덕분에 같은 과목을 여러 학기에 개설할 수 있고 재수강(같은 **course**, 다른 **offering**)이 표현된다. 결과의 **재수강** 3건이 그 증거다.

결과에 포함되는 케이스

케이스	건수	확인 요소
담당교수 미배정	4	COALESCE → '미배정'
미채점 (2026-1 진행 중)	10	CASE WHEN IS NULL → '미채점'
재수강	3	is_retake
등급 A~F 분포	—	CASE WHEN 범위 분기



```

→ scala-sql psql postgres
psql (17.10 (Homebrew))
도움말을 보려면 "help"를 입력하십시오.

postgres=#
postgres=# SELECT version()
postgres=# \conninfo
접속 정보: 데이터베이스="postgres", 사용자="muna", 소켓="/tmp", 포트="5432".
  
```

데이터베이스 목록								
이름	소유주	인코딩	로케일	제공자	Collate	Ctype	로케일	ICU 룰
dainkil	dainkil	UTF8	libc		en_US.UTF-8	en_US.UTF-8		
muna	muna	UTF8	libc		en_US.UTF-8	en_US.UTF-8		
postgres	muna	UTF8	libc		en_US.UTF-8	en_US.UTF-8		
scala_db	muna	UTF8	icu		en_US.UTF-8	en_US.UTF-8	ko-KR	
template0	muna	UTF8	libc		en_US.UTF-8	en_US.UTF-8		=c/muna +
								muna=CTc/muna
template1	muna	UTF8	libc		en_US.UTF-8	en_US.UTF-8		=c/muna +
								muna=CTc/muna

(6개 행)

```
scala_db=# SELECT current_database() AS db, current_schema() AS 현재스키마, current_user AS 접속계정 ;
\dn
```

db	현재스키마	접속계정
scala_db	app	muna

(1개 행)

스키마 목록	이름	소유주
app	muna	
public	pg_database_owner	

(2개 행)

```
SELECT
테이블
-----
courses
departments
enrollments
offering_schedules
offerings
professors
registrations
scholarships
semesters
status_changes
student_majors
students
(12개 행)
```

제약종류	개수
CHECK	25
FOREIGN KEY	15
PRIMARY KEY	12
UNIQUE	11

(4개 행)

테 이 블	건 수
01 semesters	12
02 departments	10
03 students	20
04 professors	12
05 courses	16
06 student_majors	28
07 status_changes	12
08 offerings	20
09 offering_schedules	30
10 enrollments	61
11 registrations	30
12 scholarships	16

(12개 행)

학 생 수	주 전 공 _정 상	주 전 공 _이 상
20	20	0

(1개 행)

항 목	건 수
미 채 점 성 적 (enrollments.score)	13
담 당 교 수 미 배 정 (offerings.professor_id)	2
미 납 (registrations.paid_on)	3
사 유 미 기 재 (status_changes.reason)	3
장 학 미 수 혜 학 생 (행 부 재)	9

(5개 행)

```
scala_db=# SELECT student_no AS 학번 ,
           name      AS 성명 ,
           phone     AS 연락처 ,
           address   AS 주소
FROM app.students
WHERE phone IS NULL
      OR address IS NULL
ORDER BY student_no;
```

학 번	성 명	연 락 처	주 소
20220003	박도윤	010-1122-3344	부산시 해운대구
20230002	조은우		
20230003	윤채원		
20240003	배준호		
20250003	권나은		

(5개 행)

학번	성명	연락처	입학시기	입학월초	재학연차	학적구분	수료연한
20210001	유 하 랍	010-2244-6688	2021년 03월	2021-03-01	5	졸업	수료연한 도래
20220001	김 서 준	010-2311-4521	2022년 03월	2022-03-01	4	졸업예정	수료연한 도래
20220002	이 하 은	010-4412-8830	2022년 03월	2022-03-01	4	졸업예정	수료연한 도래
20220003	박 도 윤	미 기 재	2022년 03월	2022-03-01	4	휴학중	수료연한 도래
20220004	최 지 우	010-7788-1122	2022년 03월	2022-03-01	4	졸업예정	수료연한 도래
20220005	정 민 준	010-3344-9911	2022년 03월	2022-03-01	4	졸업	수료연한 도래
20230001	강 수 아	010-5566-2233	2023년 03월	2023-03-01	3	재학중	재학기간 내
20230002	조 은 우	010-1122-3344	2023년 03월	2023-03-01	3	재학중	재학기간 내
20230003	윤 채 원	미 기 재	2023년 03월	2023-03-01	3	휴학중	재학기간 내
20230004	임 지 호	010-6677-8899	2023년 03월	2023-03-01	3	재학중	재학기간 내
20230005	한 서 연	010-9900-1234	2023년 03월	2023-03-01	3	재학중	재학기간 내
20240001	오 시 우	010-2233-4455	2024년 03월	2024-03-01	2	재학중	재학기간 내
20240002	신 아 름	010-3355-7788	2024년 03월	2024-03-01	2	재학중	재학기간 내
20240004	문 하 윤	010-4466-2211	2024년 03월	2024-03-01	2	재학중	재학기간 내
20240005	서 건 우	010-7799-3322	2024년 03월	2024-03-01	2	재학중	재학기간 내
20250001	남 유 진	010-1133-5577	2025년 03월	2025-03-01	1	재학중	재학기간 내
20250002	홍 시 윤	010-8822-4466	2025년 03월	2025-03-01	1	재학중	재학기간 내
20250003	권 나 은	미 기 재	2025년 03월	2025-03-01	1	재학중	재학기간 내
20250004	장 태 민	010-9911-2233	2025년 03월	2025-03-01	1	재학중	재학기간 내

(19개 행)

```

scala_db=# SELECT student_no                                AS 학번 ,
        name                                                AS 성명 ,
        COALESCE(phone, '미 기재 ')                        AS 연락처 ,
        TO_CHAR(admitted_on, 'YYYY"년" MM"월 "')          AS 입학시기 ,
        DATE_TRUNC('month', admitted_on)::date            AS 입학월초 ,
        EXTRACT(YEAR FROM AGE(CURRENT_DATE, admitted_on)) AS 재학연차 ,
        CASE WHEN status = '재학' AND grade = 4 THEN '졸업예정'
              WHEN status = '재학'                       THEN '재학중'
              WHEN status = '휴학'                       THEN '휴학중'
              ELSE                                         '졸업'
        END                                                AS 학적구분 ,
        CASE WHEN admitted_on <= CURRENT_DATE - INTERVAL '4 years'
              THEN '수료연한 도래'
              ELSE '재학기간 내'
        END                                                AS 수료연한
FROM app.students
WHERE status <> '자퇴'
ORDER BY admitted_on, student_no;

```

학기	학번	성명	과목코드	과목명	학점	분반	담당교수	성적	등급	재수강
2025-1	20240001	오시우	CSE202	데이터베이스	3.0	1	이알고	89.00	B	-
2025-1	20240005	서건우	CSE202	데이터베이스	3.0	1	이알고	58.00	F	-
2025-1	20250002	홍시윤	CSE202	데이터베이스	3.0	1	이알고	77.00	C	-
2025-1	20220001	김서준	CSE301	운영체제	3.0	1	김정보	96.00	A	-
2025-1	20220002	이하은	CSE301	운영체제	3.0	1	김정보	84.00	B	-
2025-1	20230002	조은우	EEE101	회로이론	3.0	1	박전자	81.50	B	-
2025-1	20250004	장태민	EEE101	회로이론	3.0	1	박전자	72.00	C	-
2025-1	20240004	문하윤	GEN103	서양철학사	3.0	1	미배정	85.50	B	-
2025-1	20250001	남유진	GEN103	서양철학사	3.0	1	미배정	88.00	B	-
2025-1	20250002	홍시윤	GEN103	서양철학사	3.0	1	미배정	79.00	C	-
2025-1	20250003	권나은	GEN103	서양철학사	3.0	1	미배정	91.00	A	-
2026-1	20230001	강수아	CSE202	데이터베이스	3.0	1	김정보		미채점	재수강
2026-1	20240005	서건우	CSE202	데이터베이스	3.0	2	이알고		미채점	재수강
2026-1	20250001	남유진	CSE202	데이터베이스	3.0	2	이알고		미채점	-
2026-1	20250002	홍시윤	CSE202	데이터베이스	3.0	1	김정보		미채점	재수강
2026-1	20250003	권나은	CSE202	데이터베이스	3.0	2	이알고		미채점	-
2026-1	20250004	장태민	CSE202	데이터베이스	3.0	1	김정보		미채점	-
2026-1	20250001	남유진	GEN105	대학생활설계	1.0	1	서교양		미채점	-
2026-1	20250002	홍시윤	GEN105	대학생활설계	1.0	1	서교양		미채점	-
2026-1	20250003	권나은	GEN105	대학생활설계	1.0	1	서교양		미채점	-
2026-1	20250004	장태민	GEN105	대학생활설계	1.0	1	서교양		미채점	-

(21개 행)